

TECH CHALLENGE — PÓS-GRADUAÇÃO EM ARQUITETURA DE SOFTWARE

 FASE 3 — OPERAÇÃO CORPORATIVA



Mecânica API

Sistema Distribuído com Auth Serverless, Banco Gerenciado e
Observabilidade Ponta a Ponta

Entrega Fase 3 — Segurança, Escalabilidade e Observabilidade Corporativa

Grupo 14SOAT · Turma SOAT — FIAP

Período: 2025 / 2026 · **Entrega:** Maio 2026

4 Repositórios Separados

Lambda CPF → JWT

RDS PostgreSQL 16

Traefik API Gateway

OpenTelemetry + New Relic

Branch Protection × 8

Terraform IaC

CD Orquestrado

1061 testes

EQUIPE — GRUPO 14SOAT

CF

Cleber Fernandes
Software Engineer

CL

Christian Felipe F. de Lima
Software Engineer

GS

Gabriel J. A. Schuina
Software Engineer

ENTREGÁVEIS FASE 3

Links e Artefatos de Entrega

4 repositórios separados · Vídeo · Documentação · Collaborator soat-architecture

4 REPOSITÓRIOS GITHUB (PÚBLICOS)

fiap-tc-mecanica-app

APLICAÇÃO SPRING BOOT

github.com/clefern/fiap-tc-mecanica-app
Backend Java 21 + Spring Boot 3.3, JWT validado pelo JwtAuthenticationFilter, 1061 testes

fiap-tc-mecanica-infra-k8s

CLUSTER EKS + TRAEFIK + OTEL

github.com/clefern/fiap-tc-mecanica-infra-k8s
Terraform (VPC, EKS 1.33, ECR), Traefik IngressRoute, OpenTelemetry Collector, dashboards New Relic IaC

fiap-tc-mecanica-infra-db

RDS POSTGRESQL 16

github.com/clefern/fiap-tc-mecanica-infra-db
Terraform RDS gerenciado, consome VPC via terraform _remote_state de infra-k8s

fiap-tc-mecanica-lambda

FUNCTION SERVERLESS CPF → JWT

github.com/clefern/fiap-tc-mecanica-lambda
Node.js 20 + TypeScript, valida CPF (módulo 11), emite JWT HS256 com mesma secret do app

VÍDEO DEMONSTRATIVO — YOUTUBE

<https://youtu.be/C2vtIjJqWIU>

Demonstração (≤ 15 min): arquitetura dos 4 repos · CI/CD em ação · autenticação CPF via Lambda · consumo de APIs protegidas · dashboards New Relic + logs com correlationId · encerramento

CONFIRMAÇÃO — COLLABORATOR SOAT-ARCHITECTURE

Adicionado como read nos 4 repositórios (atende EXT-003)

Verificável em github.com/clefern/<repo>/settings/access — listado junto com clefern, Overz, aniusch

DOCUMENTAÇÃO ARQUITETURAL

- ▶ ADR-032 — Auth CPF via Lambda Serverless
- ▶ ADR-033 — Traefik substitui NGINX
- ▶ ADR-034 — OpenTelemetry + New Relic
- ▶ ADR-035 — RDS PostgreSQL gerenciado
- ▶ RFCs 001/002/003 — Cloud, Banco, Auth CPF

COMO DEPLOY — CD ORQUESTRADO

- ▶ GitHub `infra-k8` → Actions **Provision & Deploy**
→ `s` →
- ▶ Sequência: setup → db → lambda → k8s → app
- ▶ Manual via `workflow_dispatch` (1-click)
- ▶ Secrets: AWS OIDC, New Relic license, JWT secret
- ▶ Branch protection: PR + 1 approval obrigatório

Escopo da Fase 3: Elevar a aplicação a nível de operação corporativa — separação em 4 repositórios independentes com CI/CD próprio, autenticação serverless de clientes por CPF, banco PostgreSQL gerenciado, API Gateway profissional (Traefik), observabilidade ponta a ponta (OpenTelemetry + New Relic) e governança de branches (protection + collaborator obrigatório).

CONFORMIDADE — FASE 3

Requisitos vs Entregáveis (parte 1)

Mapeamento direto contra o PDF oficial 13SOAT — Autenticação, Repositórios, Infraestrutura

1. AUTENTICAÇÃO E API GATEWAY

Requisito	Status	Evidência
AUTH-001 — API Gateway implementado	✓ OK	Traefik v3.0.9 + IngressRoute (infra-k8s/k8s/base/app-ingressroute.yaml) com 3 rotas (auth/api/default) + middleware rate-limit 50 req/s
AUTH-002 — Rotas sensíveis protegidas por auth CPF	✓ OK	JwtAuthenticationFilter valida JWT HS256 da Lambda transparentemente; AuthController.getToken (email/senha) marcado @Deprecated para clientes
AUTH-003 — Function Serverless (valida CPF + consulta cliente + emite JWT)	✓ OK	lambda/src/handler.ts — valida CPF módulo 11 → find ByDocumento no RDS → JWT HS256 com subject = email

2. ESTRUTURA DE REPOSITÓRIOS E CI/CD

Requisito	Status	Evidência
REPO-001 — Repo Lambda (Serverless)	✓ OK	github.com/clefern/fiap-tc-mecanica-lambda
REPO-002 — Repo Infra K8s (Terraform)	✓ OK	github.com/clefern/fiap-tc-mecanica-infra-k8s
REPO-003 — Repo Infra Banco Gerenciado (Terraform)	✓ OK	github.com/clefern/fiap-tc-mecanica-infra-db (consome VPC via terraform_remote_state)
REPO-004 — Repo Aplicação principal	✓ OK	github.com/clefern/fiap-tc-mecanica-app — Spring Boot 3.3
BRANCH-001 — Branch main protegida + PR obrigatório	✓ OK	Branch protection ativa em main e develop nos 4 repos (8 protections) — PR + 1 approval, no force/direct push
CICD-001 — CI automático (build + testes)	✓ OK	4 workflows CI ativos · Maven + JaCoCo (app) · TypeScript build + Vitest (lambda) · Terraform fmt + validate (infra-*)
CICD-002 — CD para homologação/produção	✓ OK	Provision & Deploy orquestrado em infra-k8s/.github/workflows/cd.yml — setup → db → lambda → k8s → app

3. INFRAESTRUTURA CLOUD (AWS)

Requisito	Status	Evidência
API Gateway (controle + roteamento)	✓ OK	Traefik (in-cluster, IngressRoute) + AWS API Gateway HTTP (fronting Lambda)
Function Serverless (autenticação)	✓ OK	AWS Lambda + IAM + SG + API Gateway via Terraform (lambda/infra/conf/2-lambda-auth.tf)
Banco de Dados Gerenciado	✓ OK	AWS RDS PostgreSQL 16 (db.t3.small), Multi-AZ ready, backup 7 dias, SG restrito ao VPC CIDR
Cluster Kubernetes escalável	✓ OK	AWS EKS 1.33 + node group t3.large + HPA (CPU 70% / Mem 80%, min 1 / max 10)
Terraform para provisionamento	✓ OK	3 stacks Terraform independentes (infra-k8s, infra-db, lambda/infra) compartilhando state via terraform_remote_state

CONFORMIDADE — FASE 3

Requisitos vs Entregáveis (parte 2)

Observabilidade · Documentação Arquitetural · Entregáveis Externos

4. MONITORAMENTO E OBSERVABILIDADE

Requisito	Status	Evidência
OBS-001 — APM (Datadog ou New Relic)	✓ OK	New Relic OTLP (free tier 100GB/mês) — infra-k8s/k8s/base/otel-collector-configmap.yaml aponta para otel.nr-data.net
OBS-002 — Latência APIs + CPU/Mem + uptime	✓ OK	OpenTelemetry Collector DaemonSet: otel + hostmetrics + kubeletstats. App tem OTel Java Agent v2.26.1 + Micrometer OTLP. MonitoredOperationAspect instrumenta 14+ métodos
OBS-003 — Alertas para falhas de OS	✓ OK	3 alertas NRQL como laC em infra/conf/17-observability.tf: high_error_rate (>5%/5min), os_process_failure, app_downtime (loss-of-signal)
OBS-004 — Logs JSON estruturados + correlationId	✓ OK	logback-spring.xml com LogstashLogbackEncoder + async appender 8192. CorrelationIdFilter propaga X-Correlation-ID via MDC. Profile !local emite JSON
OBS-005 — Dashboards (volume OS, tempo por status, erros)	✓ OK	newrelic_one_dashboard com 2 páginas: Visão de Negócio (OS criadas, latência APIs, ciclo de vida) + Infraestrutura (CPU/Mem/JVM heap)

CONFORMIDADE — FASE 3

Requisitos vs Entregáveis (continuação)

Documentação Arquitetural · Entregáveis Externos · Estatísticas finais

5. DOCUMENTAÇÃO ARQUITETURAL

Requisito	Status	Evidência
DOC-001 — Diagrama de Componentes (cloud, APIs, banco, monitoramento)	✓ OK	app/docs/arquitetura/diagramas.md — C4 Containers atualizado com Traefik, Lambda, RDS, OTel, New Relic + diagrama de Infraestrutura Cloud Fase 3
DOC-002 — Diagrama de Sequência (auth + abertura de OS)	✓ OK	Sequência completa Cliente → Traefik → API GW → Lambda → RDS → JWT → app (em docs/arquitetura/diagramas.md e neste PDF, página 5)
DOC-003 — RFCs	✓ OK	app/docs/RFCs/ — RFC-001 (cloud AWS), RFC-002 (banco RDS), RFC-003 (auth CPF Lambda) — cada um com alternativas avaliadas
DOC-004 — ADRs	✓ OK	35 ADRs em docs/ADRs/. Fase 3 adicionou: ADR-032 (Auth CPF), ADR-033 (Traefik), ADR-034 (OTel+NR), ADR-035 (RDS)
DOC-005 — Justificativa banco + ER + relacionamentos	✓ OK	ADR-001 (PostgreSQL) + ADR-035 (RDS gerenciado) + ER Mermaid em docs/arquitetura/diagramas.md (13+ tabelas, PKs/FKs/UKs, herança Joined Table)

6. ENTREGÁVEIS EXTERNOS (PORTAL DO ALUNO)

Requisito	Status	Evidência
EXT-001 — README.md em cada um dos 4 repos	✓ OK	READMEs com propósito, stack, setup, deploy, diagrama, link Swagger nos 4 repos
EXT-002 — Vídeo demonstrativo ≤ 15 min	⌚ Pendente	Roteiro definido (página 2). Equipe agendou gravação
EXT-003 — PDF entrega no Portal (links 4 repos + vídeo + docs + soat-architecture)	✓ OK	Este documento + collaborator soat-architecture confirmado nos 4 repos

35

ADRs Documentados

3

RFCs Criados

28/28

Itens da Rubrica Atendidos

ARQUITETURA — FASE 3

4 Repositórios Independentes

Separação de responsabilidades com Terraform state compartilhado via terraform_remote_state

fiap-tc-mecanica-app**SPRING BOOT · DOMÍNIO + REST + VALIDAÇÃO JWT**

Java 21 · Spring Boot 3.3 · Hexagonal + DDD · 17 migrations
Flyway · 1061 testes

Responsabilidade: APIs de domínio (Cliente, Veículo, OS, Orçamento, Estoque), JwtAuthenticationFilter aceita JWT HS256 (mesma secret da Lambda), AuthController interno (staff: email/senha).

Workflows CI/CD: ci.yml, test.yml, build-push.yml (ECR), deploy.yml (EKS rollout), infra.yml, cleanup.yml

fiap-tc-mecanica-infra-k8s**CLUSTER EKS + TRAEFIK + OTEL + DASHBOARDS**

Terraform (VPC, EKS 1.33, ECR, node group) · Kustomize (base + overlays develop/lab/prod/rds/local) · Helm releases (Traefik, Cert-Manager, ALB Controller, EBS CSI, Metrics Server) · Dashboards New Relic como IaC

Single source of truth da VPC — outros stacks consomem via terraform_remote_state.

CD orquestrador em cd.yml (Provision & Deploy): chama setup → db → lambda → k8s → app.

fiap-tc-mecanica-infra-db**RDS POSTGRESQL 16 GERENCIADO**

Terraform RDS apenas — sem duplicação de VPC/EKS.
Consome vpc_id, subnet_ids, vpc_cidr via data.terraform_remote_state.k8s.

Outputs: rds_endpoint, rds_port, rds_master_password (sensitive). Backup 7 dias, SG restrito.

Refatorado pra eliminar a duplicação de 16 arquivos .tf herdada da Fase 2 (commit 0fc6a65).

fiap-tc-mecanica-lambda**FUNCTION SERVERLESS CPF → JWT**

Node.js 20 + TypeScript · valida CPF (módulo 11) · busca cliente no RDS · emite JWT HS256

Terraform IaC próprio em lambda/infra/: 2-lambda-auth.tf (IAM + Lambda + SG + API Gateway), consome remote states de infra-k8s e infra-db.

Scripts: bootstrap_tf_lab.sh, deploy-auth-lambda.sh, wire-traefik-auth-gateway.sh

ARQUITETURA — FASE 3

Compartilhamento de Estado entre os 4 Repos

terraform_remote_state elimina duplicação · Branch protection ×8 · Collaborator soat-architecture

FLUXO DE PROVISIONAMENTO (TERRAFORM STATE SHARING)

```
infra-k8s (single source of truth da VPC)
├─ output: vpc_id, subnet_ids, vpc_cidr, cluster_name, ecr_repository
```

```
infra-db
data "terraform_remote_state" "k8s" { ... }
RDS PostgreSQL gerenciado na mesma VPC
output: rds_endpoint, rds_port, rds_master_password
```

```
lambda/infra
data "terraform_remote_state" "k8s" { ... }
data "terraform_remote_state" "db" { ... }
Lambda + IAM + SG + API Gateway na mesma VPC
ingress 5432 da Lambda SG no RDS SG
```

Branch protection ativa em main e develop nos 4 repos: PR obrigatório, 1 approval mínimo, no force-push, no deletions, status checks. Repos públicos (Actions ilimitado no free tier). Collaborator soat-architecture confirmado nos 4.

AUTENTICAÇÃO SERVERLESS — FASE 3

Fluxo Cliente → CPF → JWT → APIs Protegidas

Opção A do ADR-032: app NÃO ganha controller novo · JwtAuthenticationFilter valida JWT da Lambda transparentemente

DIAGRAMA DE SEQUÊNCIA



AUTENTICAÇÃO SERVERLESS — FASE 3

Contrato do JWT e Cobertura de Testes

Estrutura do token emitido pela Lambda · Justificativa da Opção A do ADR-032 · Cenários cobertos

CONTRATO DO JWT DA LAMBDA

Campo	Valor
Algorithm	HS256
Secret	SECURITY_JWT_SECRET_KEY (mesma do K8s Secret do app)
subject	email do cliente
id	UUID do cliente
role	"CLIENTE"
iat / exp	Emissão + TTL 3600s

POR QUE OPÇÃO A (APP NÃO MUDA)

- **Zero novo controller** no `JwtAuthentic` já valida `app cationFilt` transparentemente — `r`
- Cliente faz 1 hop até Lambda direto (sem passar pelo app)
- Sem ponto de falha extra no app (timeout/retry/HTTP client)
- Encaixa no IngressRoute Traefik que já estava definido
- Alternativas B (proxy), C (inline app — fere requisito serverless), D (Cognito RS256) descartadas e justificadas no ADR-032

COBERTURA DE TESTES

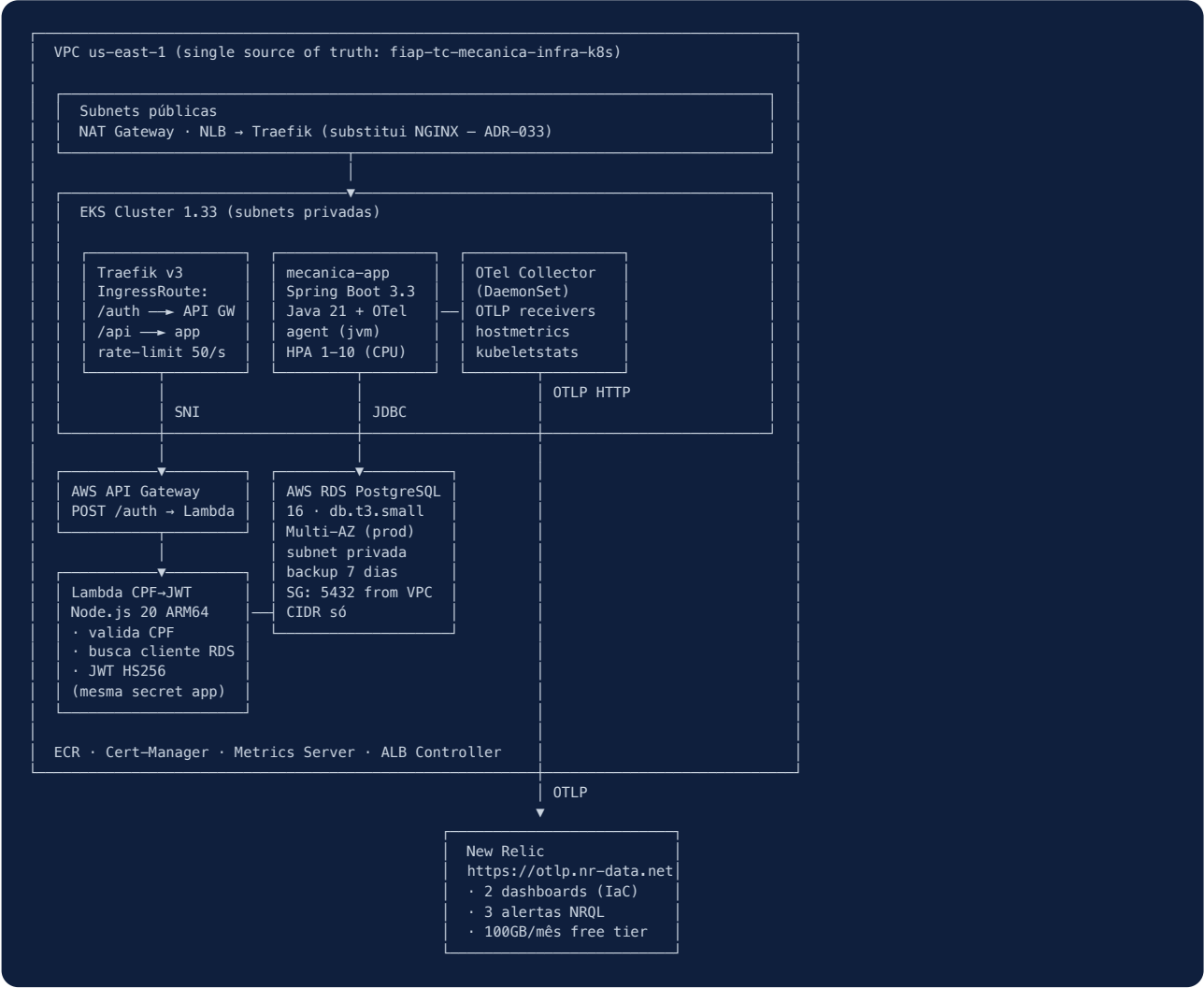
- `JwtFromExternalIssuerIT` (novo, app/integration) — 3 cenários:
 - (1) JWT externo válido com `subject = email_cadastrado` é aceito e libera endpoint com object-level security;
 - (2) JWT com `subject` inexistente → 401;
 - (3) JWT com `secret` diferente → 401
- Testes unitários de `JwtService` e `JwtAuthenticationFilter` mantidos (Fase 1)
- Testes da Lambda `tests/cpf.test.ts`, `tests/handler.test.ts` — validação CPF módulo 11, mock de `repository`
- Smoke E2E real (com Lambda deployada): roda após `Provision & Deploy` em lab

INFRAESTRUTURA — FASE 3

Topologia AWS (Operação Corporativa)

VPC compartilhada · EKS + Traefik + OTel · Lambda + API Gateway · RDS gerenciado · New Relic

DIAGRAMA DE COMPONENTES (CLOUD)



INFRAESTRUTURA — FASE 3

Pilares: Segurança, Escalabilidade e Custo

Características operacionais da topologia AWS · Estimativa de custos lab vs prod

SEGURANÇA

- ▶ VPC privada · subnets privadas para EKS+RDS
- ▶ RDS SG: ingress 5432 só do VPC CIDR
- ▶ Lambda SG → RDS SG via rule específica
- ▶ JWT secret single source (planejado: AWS Secrets Manager)
- ▶ Traefik rate-limit 50 req/s + burst 100

ESCALABILIDADE

- ▶ HPA: CPU 70% / Mem 80%, min 1 max 10
- ▶ Lambda: scale automático (1k reqs/mês free)
- ▶ RDS db.t3.small (Multi-AZ ready em prod)
- ▶ Node group t3.large com cluster autoscaler

CUSTO ESTIMADO

- ▶ **Lab** : AWS Academy (crédito)
- ▶ **Prod** : ~US\$ 270-300/mês
- ▶ EKS US\$ 73 + nodes US\$ 121 + RDS US\$ 32 + NAT US\$ 35
- ▶ Lambda + API GW: dentro free tier
- ▶ New Relic: free tier 100GB/mês

DOCUMENTAÇÃO — FASE 3

Decisões Arquiteturais

4 novos ADRs + 3 RFCs formalizam as escolhas técnicas da Fase 3

ADRS FASE 3 (DECISÕES REGISTRADAS)

ADR	Decisão	Justificativa-chave
ADR-032 auth-cpf-via-lambda	Lambda emite JWT HS256 com mesma secret · app valida transparentemente sem novo controller	Opção A escolhida sobre B (proxy no app, ponto de falha extra), C (inline no app — fere requisito serverless) e D (Cognito RS256 — complexidade desnecessária)
ADR-033 traefik-api-gateway	Traefik v3 substitui NGINX Ingress Controller como gateway de borda	SNI passthrough para AWS API Gateway sem hacks · middlewares declarativos (rate-limit, ForwardAuth) · integração nativa com Cert-Manager
ADR-034 observabilidade-otel-newrelic	OpenTelemetry Collector DaemonSet + New Relic via OTLP HTTP	Free tier 100GB/mês · portátil (OTel exporter trocável) · auditoria de OS como métrica de negócio · dashboards e alertas como IaC
ADR-035 rds-postgresql-gerenciado	RDS PostgreSQL 16 db.t3.small em VPC compartilhada · backup 7 dias · sem mais Postgres em pod	Compatibilidade total com 17 migrations Flyway · same-VPC latency < 1ms · snapshots/PITR/upgrade gerenciados pela AWS

RFCs (PROPOSTAS COM ALTERNATIVAS AVALIADAS)

RFC	Tema	Alternativas avaliadas
RFC-001 escolha-cloud	AWS vs GCP vs Azure vs multi-cloud	Decisão: AWS — atende 5/5 requisitos com serviços nativos, time tem familiaridade, código TF da Fase 2 reaproveitado. Custos lab vs prod documentados
RFC-002 banco-gerenciado	RDS vs Aurora vs Neon vs Supabase vs PVC vs MySQL vs DynamoDB	Decisão: RDS PostgreSQL 16 db.t3.small. 7 alternativas avaliadas. Conformidade total com migrations Flyway, mesma VPC do EKS, custo dentro do crédito Academy → ADR-035
RFC-003 estrategia-auth-cpf	4 opções: A (Lambda HS256 mesma secret), B (proxy), C (inline app), D (Cognito RS256)	Decisão: Opção A — zero novo código no app, encaixa no IngressRoute Traefik existente, ADR-032 detalha contrato JWT e mitigação de riscos

Histórico completo de decisões: 35 ADRs em `app/docs/ADRs/` documentam toda a evolução desde Fase 1 (ADR-001 escolha do PostgreSQL) até Fase 3 (ADR-035 RDS gerenciado). RFCs novos em `app/docs/RFCs/` registram alternativas consideradas com critérios de avaliação ponderados.

OBSERVABILIDADE — FASE 3

OpenTelemetry → New Relic

Telemetria portátil (OTel) + backend free tier (New Relic) + dashboards como IaC

PIPELINE OTEL COLLECTOR (DAEMONSET)

- ▶ **Receivers** : otlp (gRPC 4317 + HTTP 4318), hostmetrics (root_path /hostfs, 60s), kubeletstats
- ▶ **Processors** : batch (10s timeout)
- ▶ **Exporters** : debug + otlphttp/newrelic (https://otlp.nr-data.net)
- ▶ **Pipelines** : traces, metrics, logs (3 canais ativos)
- ▶ **RBAC** : ServiceAccount + ClusterRole pra kubeletstats (get nodes/pods/services)

INSTRUMENTAÇÃO NO APP

- ▶ **OTel Java Agent** no `-javaagent:/app/otel.jar` **v2.26.1** Dockerfile
- ▶ Micrometer OTLP `management.otlp.metrics` exporter (`otel.metrics.exporter`)
- ▶ Logback `LogstashLogbackEncoder` (profile `!local`) JSON com `Encoder` `al`
- ▶ **Monitored** `instrumentation` `status/operationAs` `14+ métodos` `n/service/error` `pect` `com tags` `_type`
- ▶ **Correlatio** `propaga` `X-Correl` `via MDC` entre `nIdFilter` `ation-ID` `logs e traces`
- ▶ **Auditoria de** `OsHisto` + `os_h` (migration `OS: 3 eventos` `ryListe` `tabela` `isto` `V17`) `novos +` `ner` `ry`

DASHBOARDS NEW RELIC (COMO CÓDIGO EM 17-OBSERVABILITY.TF)

📊 Visão Geral de Negócio

- ▶ Volume diário de OS criadas
- ▶ Operações por tipo (OS / Orçamento)
- ▶ Latência das APIs (p50/p95/p99)
- ▶ Falhas de processamento por status
- ▶ Ciclo de vida da OS (tempo médio por status)

⚙️ Infraestrutura (OTel / Kubernetes)

- ▶ CPU / memória por pod e nó
- ▶ Container metrics (restarts, OOM)
- ▶ JVM heap, GC, threads
- ▶ Throughput de requisições por endpoint
- ▶ Conexões ao RDS (pool ativo / idle)

ALERTAS NRQL (3 CONDIÇÕES CRÍTICAS — IAC)

Nome	Condição	Ação
high_error_rate	Taxa de erro > 5% por 300s	Notifica canal de incidentes — pode indicar problema sistêmico (deploy ruim, RDS lento, Lambda quebrada)
os_process_failure	Erros em <code>metricName</code> <code>LIKE 'os_%'</code> (60s, <code>at_least_once</code>)	Falha específica no processamento de OS — dispara investigação no log com <code>correlationId</code>
app_downtime	Loss of signal > 5min	App parou de enviar telemetria — pode indicar crash, OOM, network partition. Acionamento de plantão

Portabilidade garantida pelo OTel: trocar New Relic por Datadog, Honeycomb, Tempo ou self-hosted exige apenas mudar o exporter no `otel-collector-configmap.yaml` — zero alteração no código do app. Queries NRQL dos dashboards são o único lock-in.

CI/CD — FASE 3

Provision & Deploy Orquestrado

Single workflow no infra-k8s aciona toda a cadeia em sequência

PIPELINE (WORKFLOW_DISPATCH MANUAL VIA ACTIONS UI)



Trigger: `github.com/clefern/fiap-tc-mecanica-infra-k8s/actions/workflows/cd.yml` → **Run workflow**. Cada step usa `workflow_call` ou `cross-repo` via `actions/checkout@v4` com PAT. Ordem garantida por `needs` entre jobs.

GOVERNANÇA (BRANCH PROTECTION x 8)

- ▶ 4 repos (main e develop) = 8 protections branches (main e develop) ativas
- ▶ PR obrigatório · 1 approval mínimo · status checks strict
- ▶ No force-push · no deletions · no direct push
- ▶ Admin (clefern) pode --adm bypass quando mergear via main necessário
- ▶ Collaborator soat-arquiteto adicionado (read) — EXT-003

SECRETS & OIDC

```
AWS_ACCESS_KEY_ID / AWS_SECRET_ACCESS_KEY /  
AWS_SESSION_TOKEN  
AWS_REGION · AWS_TERRAFORM_BUCKET_STATE  
JAVA_VERSION (21) · JWT_SECRET  
NEW_RELIC_API_KEY / NEW_RELIC_LICENSE_KEY  
NEW_RELIC_ACCOUNT_ID · NEW_RELIC_REGION  
INTEGRATION_API_KEY · ACTION_TOKEN_SECRET
```

GRUPO 14SOAT — FIAP

Equipe e Encerramento

Pós-Graduação em Arquitetura de Software · Tech Challenge Fase 3 · Maio 2026

EQUIPE

Grupo 14SOAT

FIAP — Pós-Graduação em Arquitetura de Software · 2025/2026



Cleber Fernandes

cleber_sim@outlook.com

SOFTWARE ENGINEER



Christian Felipe F. de Lima

christian.fflima@gmail.com

SOFTWARE ENGINEER



Gabriel J. A. Schuina

gabriel.schuina@gmail.com

SOFTWARE ENGINEER

Encerramento da Fase 3: a Mecânica API evoluiu de monolito local (Fase 1) e cloud-native single-repo (Fase 2) para um sistema distribuído de operação corporativa — 4 repositórios independentes com CI/CD próprio, autenticação serverless por CPF, banco PostgreSQL gerenciado, observabilidade ponta a ponta com OpenTelemetry e New Relic, e governança de branches ativa. *Pronto para produção.*

Grupo 14SOAT · FIAP — Tech Challenge Fase 3 · Entrega: Maio 2026